



CIT300 Data Structures and Algorithms

Graded Practical Assignment 1 (Week 10)

University Student Record and Campus Route Management System

Coverage: Weeks 1-9 : Linear Data Structures, Trees, Hashing, and Graphs

Contribution: 10% of the final module grade

1. Important Group Requirement

Every group must complete the entire project. If your group has fewer members, combine the suggested responsibilities among the available members and still implement all specified components, including the graph component. **No component may be omitted because the group has fewer members.**

Group-member details must be recorded correctly. The project folder and README must include every member's name, student ID, assigned responsibility, and individual contribution. Missing, incomplete, or incorrect information may result in marks being deducted.

2. Submission and Google Drive Access Requirement

If the project files are too large to upload directly to the designated LMS submission link, upload the complete project to a Google Drive folder. **Copy the Google Drive folder link into a Notepad (.txt) file and upload the TXT file through the designated LMS submission link.**

Before submitting, you **MUST** give Editor access to both of the following email addresses:

- asanka.r@sltc.ac.lk
- kaushika.w@sltc.ac.lk

Check the Google Drive sharing settings before clicking Submit in the LMS. **Confirm that both email addresses have Editor access. You are responsible for verifying the permissions.**

Late emails will not be accepted. **Even if you send an email after the deadline containing the required files, Google Drive links, permissions, or access, such submissions will not be considered or accepted.** Only submissions successfully submitted through the designated LMS submission link before the deadline will be considered for marking.

Failure to provide the required Google Drive Editor permissions before submission may result in 0 marks.

3. Objective

Develop a Java console application that manages university student records and represents connections between campus locations. The system should demonstrate the practical use of linked lists, stacks, queues, trees, hashing, and graphs.

4. System Requirements

No.	Requirement
1	Store student records containing Student ID, Name, Programme, and Marks.
2	Use a linked list to store and manage student records.
3	Use a stack to maintain recent actions, deleted records, or an undo/history feature.
4	Use a queue to manage student service requests in order of arrival.
5	Use a BST or AVL tree to organize/search student records by Student ID or another suitable key.
6	Use hashing to support efficient student ID searching.

7	Use a graph to represent campus locations and their connections.
8	Represent the graph using an adjacency list or adjacency matrix.
9	Provide operations to add and remove campus locations and connections/roads.
10	Display connected locations and/or the campus network.
11	Implement at least one graph traversal: BFS or DFS.
12	Provide add, update, delete, search, and display operations for student records.
13	Provide a menu-driven console interface with suitable input validation.
14	Handle invalid inputs, duplicate IDs/locations, missing records, invalid marks, and unavailable connections appropriately.

5. Suggested Menu

1. Add Student Record
2. Update Student Record
3. Delete Student Record
4. Display All Records using Linked List
5. Add Service Request to Queue
6. Process Next Service Request
7. Display Recent Actions using Stack
8. Display Students using BST/AVL
9. Search Student using Hashing
10. Add Campus Location
11. Remove Campus Location
12. Add Campus Connection/Road
13. Remove Campus Connection/Road
14. Display Campus Connections
15. Traverse Campus Locations using BFS or DFS
16. Exit

6. Graph Component

Model the university campus as a graph. Campus locations are vertices, while roads, paths, or direct connections are edges. Use an adjacency list or adjacency matrix and provide operations for adding/removing locations and connections, displaying neighbours, and traversing the network using BFS or DFS.

Self-study: Reference videos for graphs have been provided. Although the graph component may not yet have been fully covered in lectures, students are expected to use the provided videos for self-study and include the graph component. The graph component is compulsory.

7. Suggested Team Responsibilities

Member	Suggested Responsibility
Member 1	Linked list implementation and student-record management.
Member 2	Stack and queue implementation and related operations.
Member 3	BST/AVL tree implementation and hashing/search functionality.
Member 4	Graph implementation, campus locations, connections, and BFS/DFS traversal.
All Members	Integration, validation, testing, debugging, documentation, GitHub collaboration, and completion of the entire project.

Note: These are suggested roles only. Fewer-member groups must combine roles while still completing every requirement, including graphs.

8. Implementation Guidelines

- Develop the application using Java as a console-based system.
- Implement the required data structures using appropriate classes and methods.
- Do not omit any required component or the graph functionality.
- Use meaningful names, readable code, and suitable comments.
- Validate inputs and handle invalid operations appropriately.
- Include a README with project information, complete group-member details, responsibilities, and individual contributions.
- Each member must be able to explain and demonstrate their own contribution.
- Use GitHub collaboration features such as branches, commits, and pull requests where applicable.

9. Deliverables

- GitHub repository containing the complete project.
- README with complete and correct group-member information and individual contributions.
- One merged demonstration video showing the completed system.
- Each member's contribution represented in the merged video.
- Evidence of collaboration through commits, branches, and pull requests where applicable.

10. Final Submission Checklist

- ☐ Complete project implemented, including all data structures and graph functionality.
- ☐ All group members' names and student IDs are correct.
- ☐ Responsibilities and individual contributions are documented.
- ☐ GitHub repository and demo video are complete.
- ☐ If using Google Drive, the complete project is uploaded.
- ☐ Google Drive link is copied correctly into the Notepad (.txt) file.
- ☐ asanka.r@sltc.ac.lk has Editor access.
- ☐ kaushika.w@sltc.ac.lk has Editor access.
- ☐ Google Drive permissions are checked before LMS submission.
- ☐ The submission is successfully completed through the designated LMS link before the deadline.

Deadline: Submit through the designated LMS submission link on or before the 29th September.

Remember: An email sent after the deadline does not make a late submission valid, even when files, links, permissions, or access are provided by email.